

<http://localhost:8080/game-results>

<http://localhost:8080/leaderboard>

<http://localhost:8080/leaderboard?rank=5>

Das Projekt ist ein REST-Webservice, der Spielergebnisse eines Spiels verwaltet.

Clients (z.B. Browser, Postman oder eine App) können über HTTP-Anfragen:

Neue Spielergebnisse hinzufügen, ein bestimmtes Ergebnis abrufen, alle Ergebnisse abrufen,
ein Ergebnis löschen, ein Leaderboard (Rangliste nach Score und ID) anzeigen

Die Daten werden nur im Speicher gespeichert und nicht in einer Datenbank.

SpringApp (Startklasse)

Startet die Spring Boot Anwendung.

GameResultController

Diese Klasse stellt die Spielresultate bereit.

HTTP-Anfragen werden hier empfangen und an den Service weitergegeben.

Beispiele:

- GET „/game-results“ -> alle Ergebnisse
- GET „/game-results/{id}“ -> ein Ergebnis
- POST „/game-results“ -> neues Ergebnis
- DELETE „/game-results/{id}“ -> Ergebnis löschen

Der Controller enthält keine Logik, sondern ruft nur den Service auf.

LeaderboardController

Dieser Controller liefert das Leaderboard.

Die Ergebnisse werden vom Service geholt und nach Score Absteigen und ID aufsteigend sortiert.

GameResultService

Der Service enthält die Geschäftslogik.

Spielergebnisse speichern, IDs vergeben, Ergebnisse suchen, Ergebnisse löschen, Liste der Ergebnisse zurückgeben

GameResult

Diese Klasse beschreibt ein Spielergebnis. Enthält die Daten eines Spielergebnisses.

2.2 Ändern der Implementierung

2.2.1 Sortierung

In GameResults sieht man die verschiedenen angelegten Variablen.

```
gameResultService.getGameResults().sortedWith(compareBy({ -it.score }, { it.id })))
```

ändern zu

```
gameResultService.getGameResults().sortedWith(compareBy({ -it.score }, { it.timeInSeconds })))
```

2.2.2 Zusätzliche Abfrage

Wenn rank nicht angegeben wird, soll das leaderboard gleich angezeigt werden. Wenn rank angegeben sollen der Spieler auf dem entsprechenden Platz sowie die oberen und unteren 3 anderen Spieler zurückgegeben werden.

Packages hinzufügen:

```
import org.springframework.web.server.ResponseStatusException
import org.springframework.http.HttpStatus
```

für die HTTP 400 Antwort.

Start wird inklusive mitgezählt.

Ende wird exklusive, also nicht mitgezählt.

Klasse LeaderBoardController.kt

Ist der Endpunkt von /leaderboard über den man das ganze leaderboard abrufen kann oder nur die jeweiligen Spieler, wenn Rank angegeben ist. Ungültige Ränge geben einen WhiteLabelErrorPage aus http-400.

```
if(rank == null){  
    return leaderboardSorted;  
}
```

Ganze Liste/Leaderboard wird ausgegeben.

```
if(rank < 1 || rank > leaderboardSorted.size){  
    throw ResponseStatusException(HttpStatus.BAD_REQUEST)  
}
```

prüft ob rang gültig ist. Gibt den http 400 Fehler aus damit man weiß dass der rang ungültig/falsch ist.

```
val index = rank - 1 //Ränge starten bei 1, Indexe einer Liste jedoch bei 0  
val start = maxOf(0, index - 3) //3 Spieler vor dem Rang  
val end = minOf(leaderboardSorted.size, index + 4) //3 Spieler über dem rang, bei +3 werden nur 2  
Spieler drüber angezeigt  
return leaderboardSorted.subList(start, end) //gibt die Spieler um den Rang zurück +-3
```

Klasse LeaderBoardController.Tests.kt

Testes den Controller, ohne den Server starten zu müssen. Verwendet Mockito um alles zu simulieren.

@Test

```
fun test_getLeaderboard_correctScoreSortingRankNOtThere() {
    val a = GameResult(1, "A", 100, 20.0)
    val b = GameResult(2, "B", 90, 15.0)
    val c = GameResult(3, "C", 90, 14.0)

    whenever(mockedService.getGameResults()).thenReturn(listOf(a, b, c))

    val res = controller.getLeaderboard(null)

    verify(mockedService).getGameResults()
    assertEquals(listOf(a, c, b), res)
}
```

Testet die Sortierung des Leaderboards nach dem Scoring und Zeit, wenn kein Rang angegeben ist.

@Test

```
fun test_getLeaderboard_rankAroundPlayers() {
    val players = listOf(
        GameResult(1, "Anna", 150, 32.5),
        GameResult(2, "Chris", 150, 35.0),
        GameResult(3, "Eli", 140, 33.5),
        GameResult(4, "Ben", 120, 30.0),
        GameResult(5, "Dora", 100, 28.0),
        GameResult(6, "Muje", 100, 28.0),
    )

    whenever(mockedService.getGameResults()).thenReturn(players)

    val res = controller.getLeaderboard(5) // Rank = 5

    assertEquals(listOf(players[1], players[2], players[3], players[4], players[5]), res)
}
```

Prüft ob Rank 5 +- 3 Spieler beim LeaderBoard anzeigen korrekt ausgegeben wird.

```

@Test
fun test_getLeaderboard_sameScore_CorrectIdSorting() {
    val first = GameResult(1, "first", 20, 9.0)
    val second = GameResult(2, "second", 20, 10.0)
    val third = GameResult(3, "third", 20, 15.0)

    whenever(mockedService.getGameResults()).thenReturn(listOf(first, second, third))

    val res: List<GameResult> = controller.getLeaderboard(null)

    verify(mockedService).getGameResults()
    assertEquals(3, res.size)
    assertEquals(first, res[0])
    assertEquals(second, res[1])
    assertEquals(third, res[2])
}

```

Ursprüngliche Klasse, nur halt wegen `timeInSeconds` Änderung bisschen modifiziert.

2.2.3 Zusätzliche Abfrage

Bei Klasse `LeaderBoardControllerTest` wird noch nicht getestet ob bei ungültigem Rang die Klasse korrekt ausgeführt wird. Es fehlt noch `rank > leaderboardSorted.size` ein `true` hit und dann ist die `coverage = 100%`

```

@Test
fun test_getLeaderboard_rankTooHigh_throwsBadRequest() {
    val players = listOf(
        GameResult(1, "A", 100, 20.0),
        GameResult(2, "B", 90, 15.0),
        GameResult(3, "C", 80, 10.0)
    )

    whenever(mockedService.getGameResults()).thenReturn(players)

    // Rank 5 ist größer als die Anzahl der Spieler, sollte HTTP 400 werfen
    assertFailsWith<ResponseStatusException> {
        controller.getLeaderboard(5)
    }
}

```

```
}
```

Noch hinzugefügt. Rank 5 ist größer als da leaderboard size

Bei Testklasse GameResultServiceTests gibt es fehler

Wenn man init ergebnisse nicht macht, sind die tests korrekt. Bei GmeResultService die Inits wieder hinzufügen, wenn man es auf dem Web anzeigen lassen will.

`fun deleteGameResult(id: Long) = gameResults.removeIf { it.id == id }` wird noch nicht getestet

Man muss ein GameResult hinzufügen, ihn löschen und dann prüfen, ob er tatsächlich weg ist.

```
@Test
fun test_deleteGameResult_existingId_removesElement() {
    val gameResult = GameResult(0, "player1", 17, 15.5)

    service.addGameResult(gameResult)
    service.deleteGameResult(1)

    val res = service.getGameResults()

    assertEquals(0, res.size)
}
```

Um eine Code Coverage im GameController von 100% zu erreichen, muss man eine eigene Testklasse GameControllerTests erstellen und Games hinzufügen, Liste von Games hinzufügen, versuchen ein game zu adden und versuchen ein Game zu löschen

verify kommt aus Mockito, dem Mocking-Framework.

```
verify(mockedService).getGameResults()
```

Prüft, ob die Methode getGameResults() tatsächlich auf dem Mock-Objekt mockedService aufgerufen wurde.

Wenn die Methode nicht aufgerufen wird, schlägt der Test fehl.

Es testet nicht das Ergebnis, sondern die Interaktion zwischen Controller und Service.

```
@Test
```

```
fun test_getAllGameResults_returnsWholeListGameResult() {
```

Hat noch eine Fehlermeldung weil eine Liste erwartet wurde.

Behebung: `val list = listOf(a, b, c)`